

PROJECT DOCUMENTATION

ClipMind AI

AI-Powered Video Summarization & Key Moments Detection Platform

Submitted By

Dipanshi Verma

Program

Infosys Springboard Internship

Duration

June 29, 2026 – August 29, 2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. Problem Statement	3
3. Objectives	3
4. Proposed System	4
5. System Architecture	5
6. Modules and Features	7
7. Technology Stack	10
8. Methodology and Workflow	10
9. Implementation	11
10. Design Considerations & Challenges	12
11. Future Scope	12
12. Conclusion	13
13. References	13

1. Introduction

The volume of long-form video available online and inside organisations — lectures, meetings, interviews, and podcasts — continues to grow faster than anyone's ability to sit through it. Video is an effective way to share information, but reviewing it end-to-end to find one relevant section is slow: there is no equivalent of skimming a page or Ctrl+F searching a transcript.

ClipMind AI is a video-intelligence platform built to close that gap. A user uploads a video, and the platform automatically extracts the audio, transcribes the speech, produces a concise summary, and surfaces the moments most likely to matter — each anchored to a timestamp in the original recording. The result is a video that can be skimmed in minutes instead of watched in full.

The platform is organised around four user roles — Content Creator, Learner, Educator, and Administrator — each with a distinct view of the same underlying content, and is built as a conventional two-tier web application: a React single-page frontend talking to a FastAPI backend over a REST API.

2. Problem Statement

Educational lectures, internal meetings, interviews, and podcasts routinely run from thirty minutes to several hours. Locating a specific topic, decision, or quote inside that runtime today means manually scrubbing the timeline or relying on whatever notes were taken live — both slow and error-prone, and neither scales as the volume of recorded video grows.

There is a clear need for a system that can, without human intervention: convert spoken content to text, distill that text into an accurate summary, and identify the timestamps of the segments that carry the most information — while keeping the process fast and inexpensive enough to run on ordinary hardware rather than requiring a dedicated GPU cluster or a paid third-party API for every video.

ClipMind AI addresses this by combining video upload and management, speech-to-text transcription, extractive summarization, key-moment detection, and analytics into a single authenticated platform.

3. Objectives

- Provide a secure, authenticated platform for uploading, storing, and managing video and audio files.
- Transcribe spoken content automatically using an offline, CPU-friendly speech-recognition model.
- Persist transcripts so they can be re-used by every downstream processing stage without re-running transcription.
- Generate accurate, concise summaries directly from the transcript, without depending on a paid external AI API.
- Detect and rank key moments and important segments across a video's timeline.
- Extract representative keywords to give a quick, scannable view of each video's topics.
- Provide an analytics dashboard covering speaker share, keyword trends, and a sentiment timeline.
- Support four distinct user roles — Content Creator, Learner, Educator, Administrator — each scoped to the right level of access.

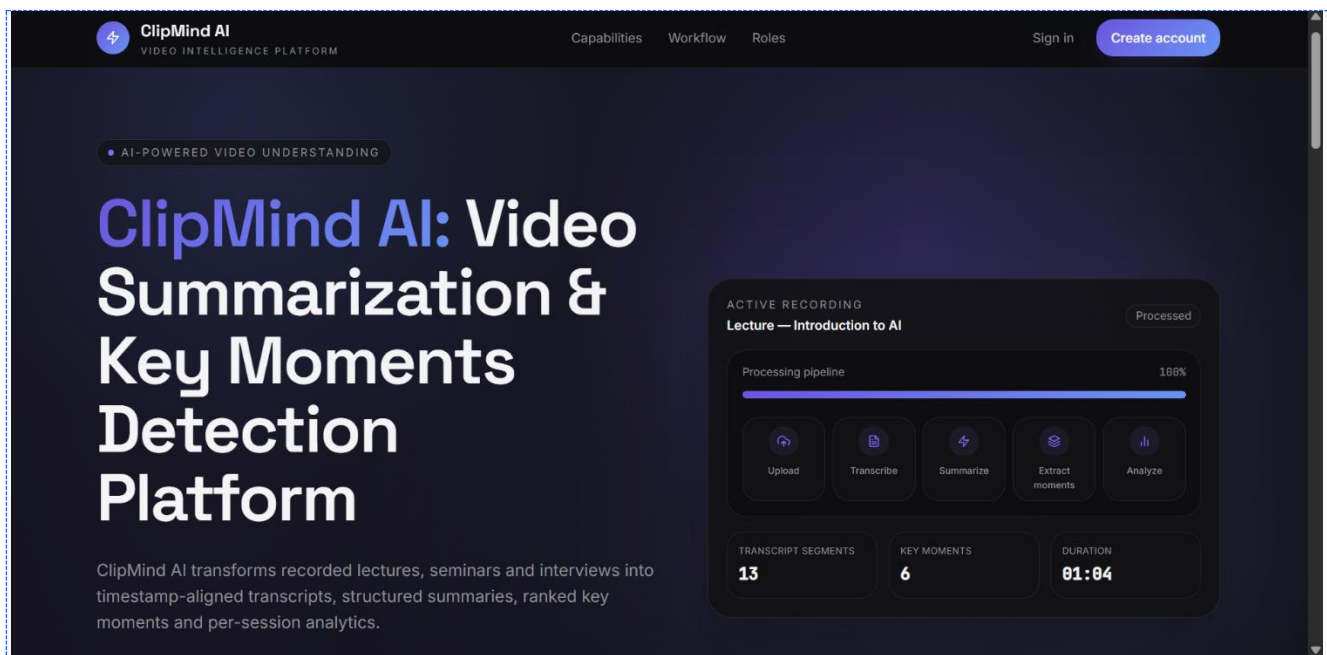
- Reduce the time it takes a viewer to find and understand the relevant parts of a long video.

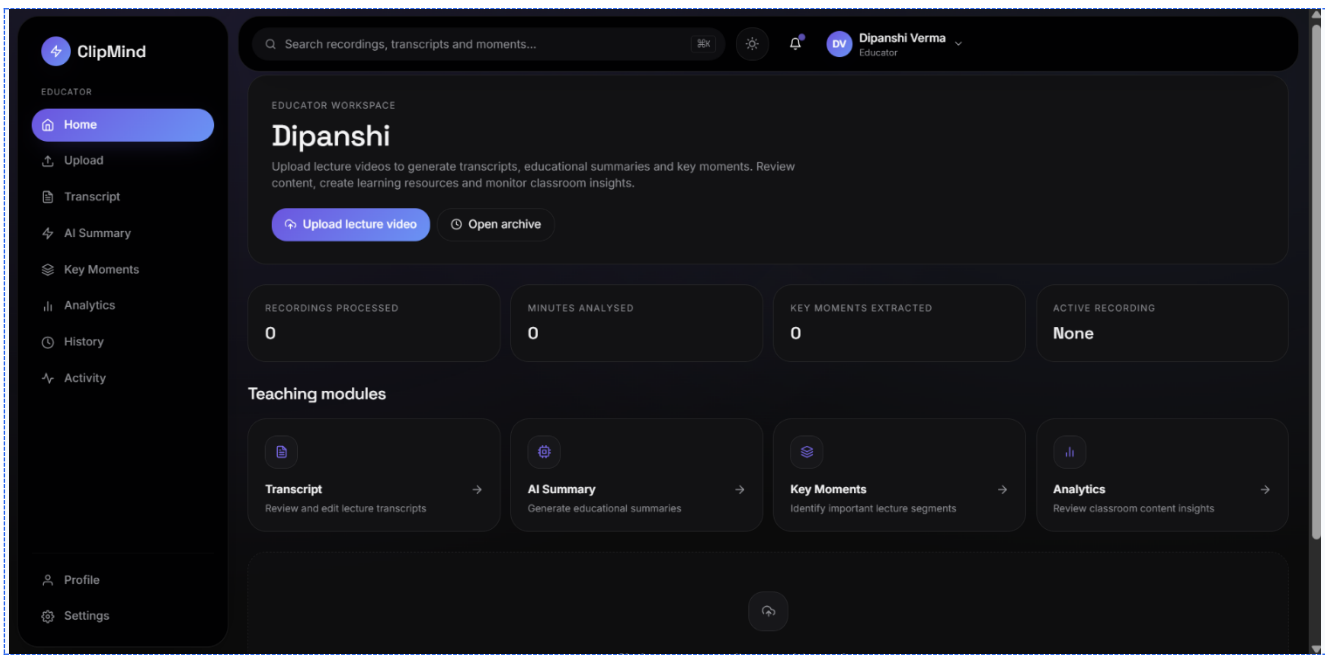
4. Proposed System

ClipMind AI is implemented as a two-tier web application. The frontend is a React 19 single-page application built with TanStack Start and TanStack Router for file-based routing, styled with Tailwind CSS v4 and shadcn/ui components on top of Radix primitives. The backend is a FastAPI service backed by SQLAlchemy and SQLite, exposing a REST API consumed by the frontend through Axios.

Processing follows a fixed, dependency-ordered pipeline: once a video is uploaded, the backend extracts audio with FFmpeg, transcribes it with faster-whisper, generates an extractive summary and keyword list from the transcript, scores and ranks key moments across the transcript's timeline, and finally computes analytics (speaker share, keyword frequency, and a lexicon-based sentiment timeline) from the transcript and the moments together.

The overall data flow can be summarised as:





User registers / logs in → Uploads a video → Backend validates and stores the file → FFmpeg extracts audio → faster-whisper transcribes it → TextRank-based summarization runs on the transcript → Key moments are scored and ranked → Analytics are computed → Results are persisted and served back to the frontend for the user to browse, search, and download.

5. System Architecture

The system follows a conventional client–server design with a clean separation between the presentation layer, the API layer, and the processing layer.

5.1 Client / Presentation Layer

The React frontend provides authenticated, role-aware pages: a dashboard, a video upload flow, transcript / summary / key-moments viewers, an analytics dashboard, upload history, activity logs, and (for Administrators) user and platform-settings screens. TanStack Router's file-based convention keeps each page and its data-loading logic co-located under `src/routes`.

5.2 API / Service Layer

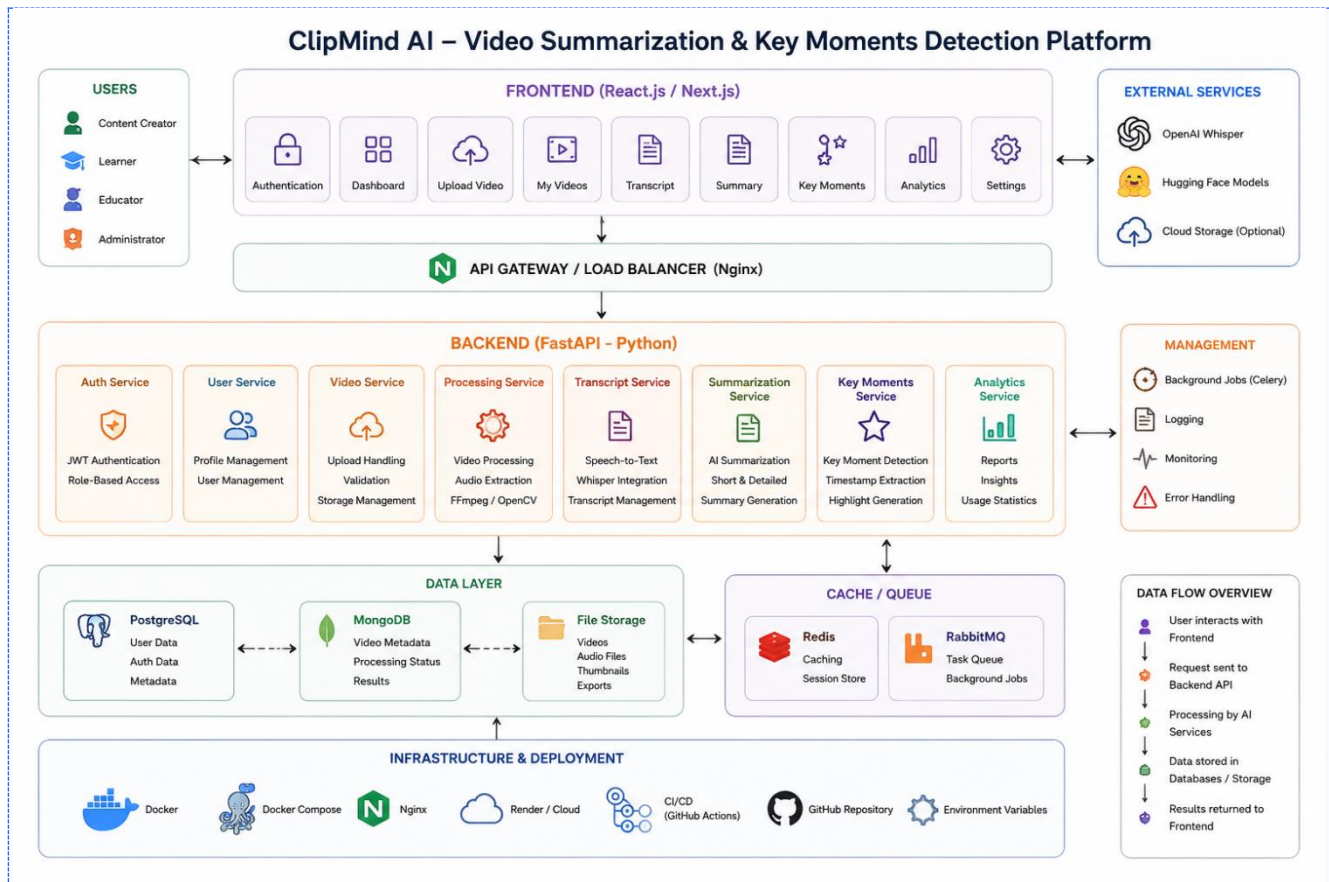
FastAPI exposes all functionality under a single `/api` prefix, organised into one router per concern — auth, videos, transcript, summary, moments, analytics, history, activity, and search. Each router delegates business logic to a matching service module, keeping request/response handling separate from processing logic.

5.3 Processing Layer

Video processing is handled by dedicated services: `ffmpeg_service` (audio extraction and duration probing), `whisper_service` (speech-to-text via faster-whisper), `nlp_service` (TF-IDF + TextRank summarization and keyword extraction), `moments_service` (key-moment scoring), and `analytics_service` (speaker share, keyword frequency, sentiment timeline). A pipeline module sequences these stages — transcription, then summary, then key moments, then analytics — and persists each stage's output before the next one runs.

5.4 Data Layer

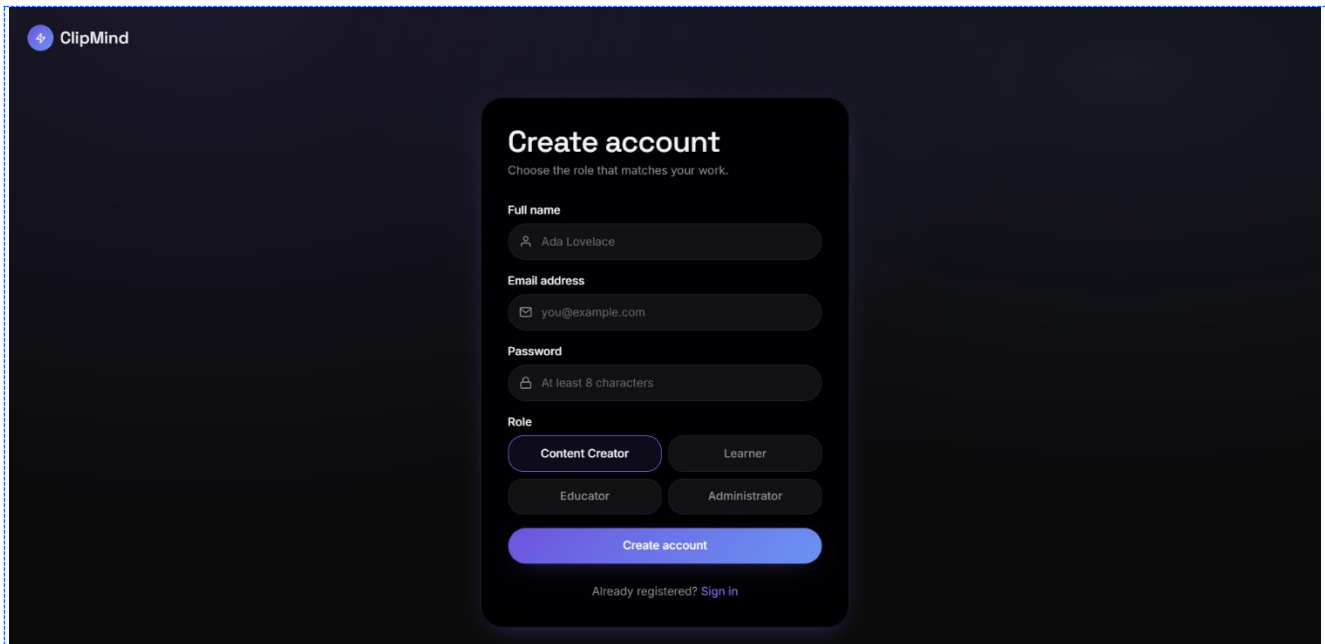
Application state is stored in SQLite via SQLAlchemy: a Users table (with a role column driving access control) and a Videos table that stores file paths, processing status, and the transcript, summary, moments, and analytics results as JSON columns keyed to each video.



6. Modules and Features

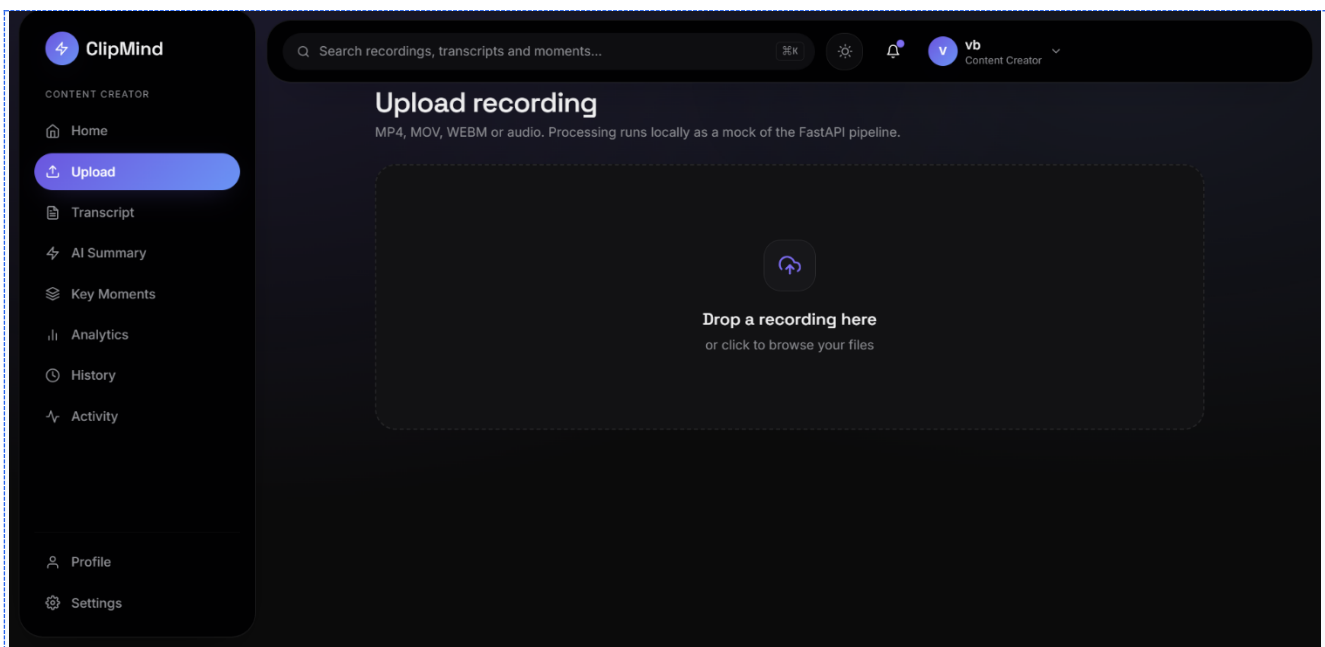
6.1 User Management Module

Registration, login, and forgot-password flows are backed by JWT authentication with PBKDF2-hashed passwords. Every account carries a role — Content Creator, Learner, Educator, or Administrator — set at signup and used to scope what a user can see and do.



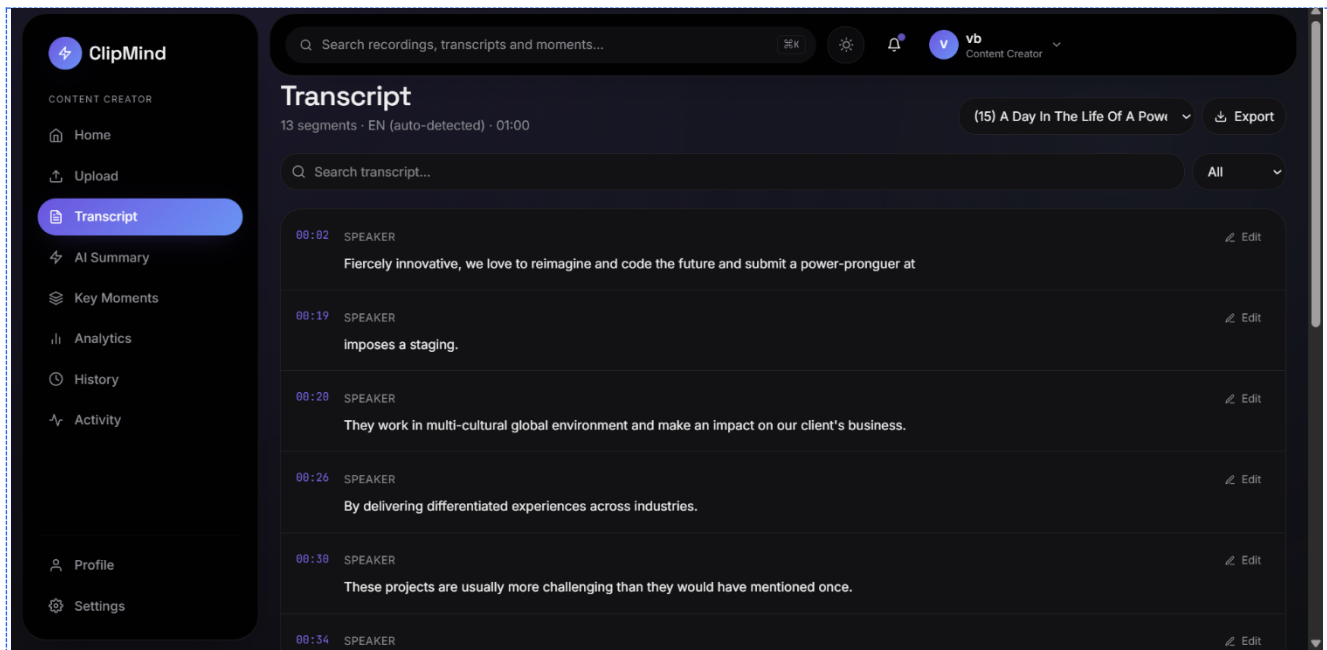
6.2 Video Upload Module

Users upload video or audio files (validated by extension and size against a configurable limit) which are stored on disk and registered in the database with a Queued status, ready for processing.



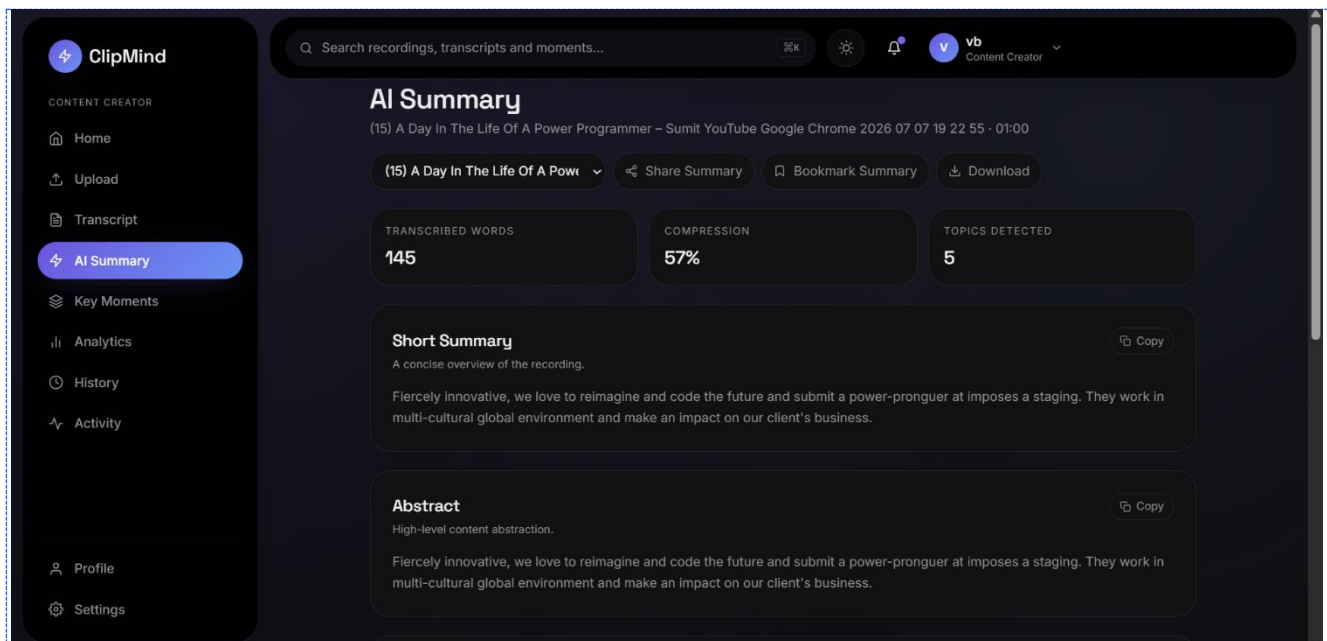
6.3 Transcript Generation Module

Audio is extracted from the uploaded file with FFmpeg and transcribed with faster-whisper, entirely offline once the chosen model has been downloaded once. The resulting timestamped segments are stored as the video's transcript and reused by every later stage.



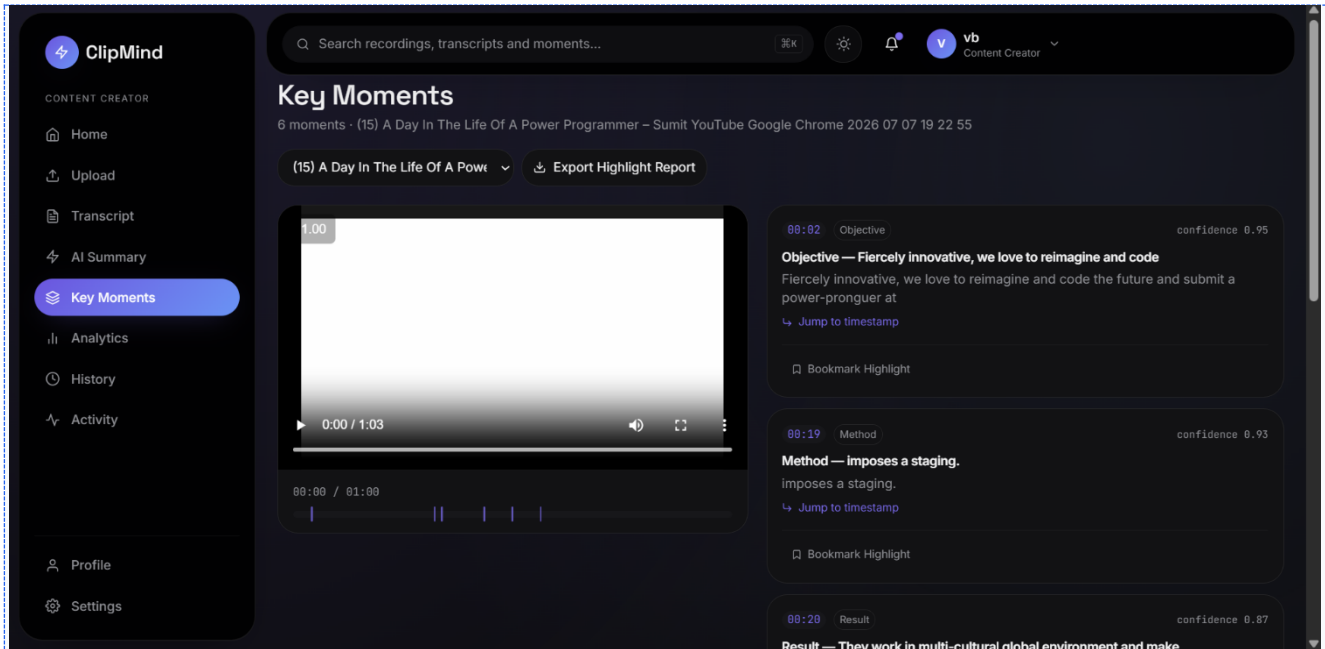
6.4 Video Summarization Module

The transcript is summarised with an extractive TextRank algorithm: sentences are embedded with TF-IDF, scored for centrality via cosine similarity and PageRank, and the highest-scoring sentences are returned in their original order — producing a summary built entirely from the speaker's own words, with no external model download required.



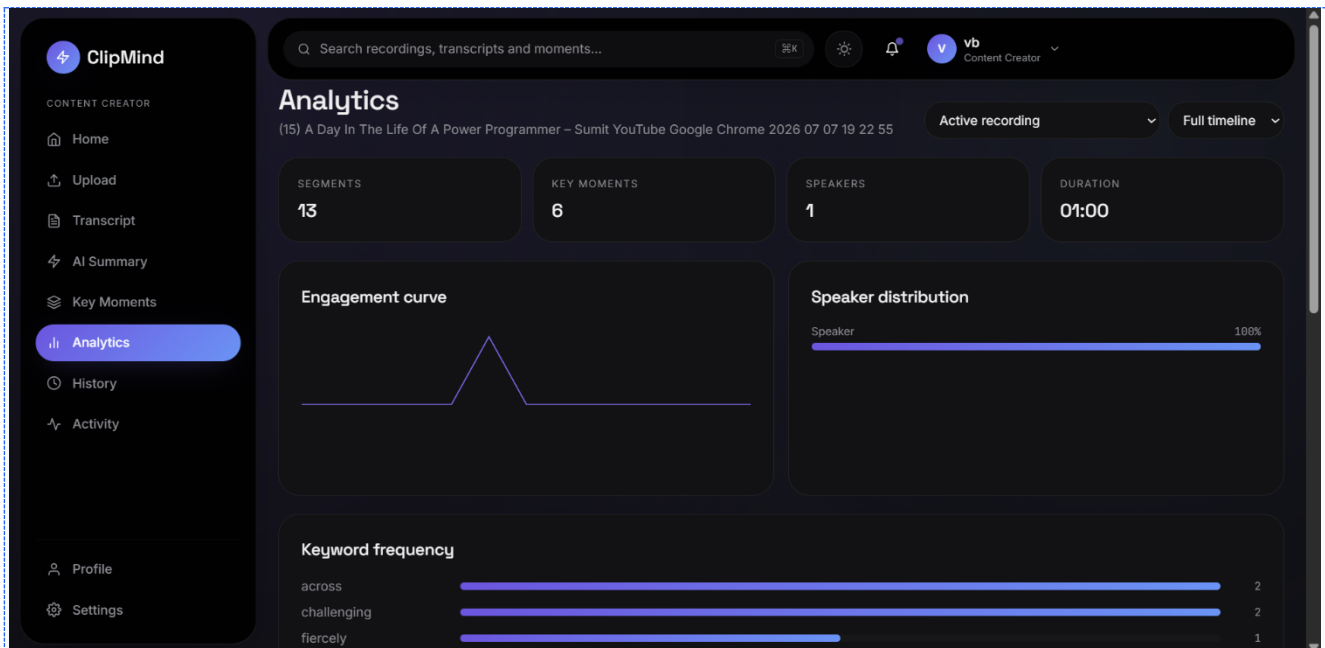
6.5 Key Moments Detection Module

Each transcript segment is scored on a blend of TextRank salience, keyword density, and a small positional bonus for openings and closings (which tend to carry framing and conclusions). The top-scoring segments are tagged (Objective, Method, Result, Discussion, Q&A, Conclusion), given a confidence score, and returned in chronological order with their timestamps.



6.6 Analytics Dashboard Module

Per-video analytics cover speaker share (segment distribution across detected speakers), the most frequent keywords, a lexicon-based sentiment timeline across the video's duration, and headline metrics (segment count, key-moment count, speaker count, and duration).



6.7 History, Search & Activity Modules

An upload-history view lists a user's past videos with filtering by status, date range, and free-text search across titles, filenames, and transcript content. An activity log records key actions per user; Administrators can view activity across every account, while other roles see only their own.

7. Technology Stack

Category	Technology
Frontend framework	React 19 with TanStack Start & TanStack Router (file-based routing)
Build tooling	Vite + TypeScript
UI / styling	Tailwind CSS v4, shadcn/ui (Radix primitives), Framer Motion
Frontend data / forms	TanStack Query, React Hook Form + Zod, Axios, Recharts
Backend framework	Python 3.11, FastAPI, Uvicorn
ORM / database	SQLAlchemy, SQLite
Speech-to-text	faster-whisper (CTranslate2-based Whisper implementation)
Summarization & keywords	scikit-learn (TF-IDF) + NetworkX (TextRank / PageRank)
Media processing	FFmpeg (audio extraction, duration probing)
Authentication	PyJWT (HS256) with PBKDF2-HMAC password hashing
Containerization	Docker (backend)
Hosting	Render (frontend and backend)
Tooling	Git & GitHub, VS Code, Postman

8. Methodology and Workflow

Processing runs as four ordered stages, each of which persists its output before the next stage is allowed to start — enforced both by the backend pipeline and by the frontend, which disables a stage's controls until its prerequisite has completed.

8.1 Transcription

If no audio has been extracted yet for a video, `ffmpeg_service` pulls a mono WAV track from the source file. `whisper_service` then runs `faster-whisper` over that audio (model size, device, and compute type are configurable via `WHISPER_MODEL_SIZE`, `WHISPER_DEVICE`, and `WHISPER_COMPUTE_TYPE`, defaulting to a small CPU-friendly int8 model), producing timestamped segments and a detected language.

8.2 Summarization

The full transcript text is split into sentences, vectorised with TF-IDF, and scored for centrality using cosine similarity and PageRank (TextRank). The highest-ranked sentences are returned in their original transcript order to form the summary, alongside a frequency-based keyword list.

8.3 Key Moment Detection

Each transcript segment is scored as a weighted combination of its TextRank salience, its density of high-value keywords, and a small bonus for position near the start or end of the video. The top-N segments are selected, tagged by role (Objective, Method, Result, Discussion, Q&A, Conclusion), and returned sorted back into chronological order.

8.4 Analytics

Speaker share is computed from segment-level speaker labels, keyword frequency from the full transcript, and a sentiment timeline from a lightweight positive/negative lexicon applied per segment — combined with the key-moments output into a single analytics payload.

9. Implementation

9.1 Project Structure

ClipMind-AI/	
src/	Frontend (TanStack Start)
routes/	File-based routes
_authenticated/	Protected: dashboard, upload, transcript, summary, moments, analytics, history, users
components/	shadcn/ui-based components
services/	auth, video, transcript, summary, moments, analytics, history API clients
context/	Auth, Theme, Workspace, Toast providers
clipmind-backend/	Backend (FastAPI)
main.py	App entrypoint, CORS, router registration
config.py	Environment-driven settings
models/	SQLAlchemy models + Pydantic schemas
routes/	auth, videos, transcript, summary, moments, analytics, history, activity, search
services/	ffmpeg, whisper, nlp, moments, analytics, activity, pipeline
data/	uploads/, audio/, clipmind.db (auto-created)
Dockerfile	

9.2 Frontend Implementation

The frontend follows TanStack Start's file-based routing convention: every file under `src/routes/` becomes a page, with a dedicated `_authenticated` layout guarding pages that require a logged-in session. Each domain (videos, transcript, summary, moments, analytics, history) has its own service module in `src/services/` that wraps a shared Axios instance (`src/services/api.ts`), which attaches the JWT from the auth context to every request.

9.3 Backend Implementation

The backend mounts one APIRouter per domain under a shared `/api` prefix in `main.py`, with CORS configured for the local Vite dev server and the deployed Render frontend. Each router is a thin layer over its matching service module — for example, `routes/videos.py` handles upload validation and delegates persistence to the SQLAlchemy Video model, while `routes/auth.py` issues and verifies JWTs and hashes passwords with PBKDF2.

9.4 The Processing Pipeline

A single pipeline module (`services/pipeline.py`) exposes `run_transcription`, `run_summary`, `run_moments`, and `run_analytics`, plus a `run_full_pipeline` convenience function that runs all four in order. Each function commits

its result to the database and refreshes the Video row before returning, so a partially completed pipeline always leaves a consistent, resumable state.

9.5 Containerization

The backend ships with a Dockerfile based on python:3.11-slim, installing FFmpeg and the Python dependencies from requirements.txt before copying the application code and exposing port 8000 behind Uvicorn. The frontend is deployed separately as its own Render web service.

10. Design Considerations & Challenges

10.1 Enforcing the Processing Order

The backend only allows summary generation once a transcript exists, moment detection once a summary exists, and so on. The frontend mirrors this by disabling each stage's controls until its prerequisite has finished, so a user cannot trigger a request the backend would reject, and the dependency chain is visible rather than a silent backend-only rule.

10.2 Choosing Extractive Summarization Over a Hosted LLM

Rather than depending on a paid third-party summarization API, ClipMind AI uses TF-IDF plus TextRank — a lightweight, deterministic, fully offline approach. The trade-off is a summary built entirely from the transcript's own sentences rather than freely-generated prose, in exchange for zero external cost, no rate limits, and predictable behaviour on CPU-only hardware.

10.3 Whisper Model Size vs. Processing Time

faster-whisper's model size is configurable (WHISPER_MODEL_SIZE), trading transcription accuracy against processing time and memory. Running on CPU with int8 compute keeps the platform usable on modest hardware, at the cost of slower turnaround on longer videos than a GPU-backed deployment would achieve.

10.4 Local Storage vs. Managed Storage

Uploaded media, extracted audio, and the SQLite database currently live on local disk, which is simple to run and reason about but does not survive a redeploy on most hosting platforms and does not scale horizontally — the natural next step being to move uploads to object storage and the database to a managed instance (see Future Scope).

11. Future Scope

- Migrate from SQLite to PostgreSQL for production-scale, concurrent deployments.
- Move uploaded media and extracted audio to object storage (e.g. an S3-compatible bucket) instead of local disk.
- Add multi-language transcription and near-real-time transcription for live sessions.
- Layer speaker identification on top of the existing speaker-share analytics.
- Expand role-based permissions beyond the current single role field on each user.
- Explore personalised summaries and recommendations based on a user's learning history and bookmarks.
- Ship a mobile application for on-the-go upload and review.

12. Conclusion

ClipMind AI demonstrates a complete, working pipeline from raw video to structured, searchable information — combining offline speech recognition, classical NLP techniques, and a role-aware full-stack web application. Choosing extractive, deterministic methods (TF-IDF/TextRank) over a hosted generative model kept the system self-contained, inexpensive to run, and predictable, while the FastAPI/React separation and the ordered processing pipeline made it possible to build, test, and reason about each stage independently.

The platform, as it stands, is a solid MVP: authentication, upload, transcription, summarization, key-moment detection, and analytics are all functioning end to end. The clearest next steps are largely about production-readiness — a managed database, object storage for media, and broader role-based permissions — rather than new core functionality.

13. References

- faster-whisper — CTranslate2-based implementation of OpenAI's Whisper speech-recognition model.
- FastAPI Documentation — <https://fastapi.tiangolo.com>
- FFmpeg Documentation — <https://ffmpeg.org/documentation.html>
- scikit-learn Documentation — <https://scikit-learn.org>
- NetworkX Documentation — <https://networkx.org>
- TanStack Start / TanStack Router Documentation — <https://tanstack.com>
- Render Documentation — <https://render.com/docs>
- Project repository — github.com/dipanshiverma/ClipMind-AI